

Computational Identity: State of the Art

Physical implementation, algorithm identity, abstraction, actual execution,
and functionalism

Contextual literature review for the Computational Identity research package

August 2026

Scope and purpose

This literature review situates the package’s proposed theory within the mature fields from which it draws: physical implementation and triviality, the distinction between functions and algorithms, operational semantics and behavioral equivalence, abstraction and causal abstraction, event-based accounts of actual execution, provenance and actual causation, and computational functionalism in consciousness research.

The central historical fact is that these problems have mostly been studied in separate communities. Philosophy of computation asks when a physical system implements a computation; semantics asks when programs or processes are behaviorally equivalent; algorithm theory asks what an algorithm is beyond the function it computes; abstract interpretation organizes loss of detail; causal abstraction asks when a high-level causal model is faithfully realized by a low-level one; concurrency and provenance study the structure of particular executions. The package’s distinctive move is to combine these ingredients into one ontology of computational realization and then use that ontology to discipline computational functionalism.

1 The implementation problem: when does a physical system compute?

The modern philosophical problem begins from a simple tension. Computation appears physically instantiated: a processor, brain, relay network, or mechanical calculator can objectively succeed or fail at a specified computation. Yet an abstract computation is not itself a physical object. Some relation must connect physical states and transitions to abstract states and transitions. If that relation is allowed to be an arbitrary mapping, the notion threatens to become trivial.

Putnam’s permutation and realization arguments helped crystallize the worry that sufficiently liberal mappings can make ordinary physical systems appear to implement arbitrary finite automata [31]. Chalmers responded by requiring an implementation relation that respects causal structure rather than merely matching a sequence of states after the fact [7, 8]. His combinatorial-state account is especially relevant because computational states are decomposed into substates with values, and physical causal relations are required to mirror the transition structure of the abstract automaton. This makes implementation a claim about organized counterfactual transition structure, not a post-hoc renaming of a history.

Sprevak’s discussion of triviality arguments emphasizes that the problem is not eliminated simply by insisting that there is some mapping from physical to computational states: the admissibility conditions on mappings do substantive work [34, 35]. More recent work continues to debate how determinate computational implementation really is. Fresco, Copeland, and Wolf defend a form of indeterminacy [16]; Curtis-Trudel argues for more determinate accounts based on resemblance and implementation constraints [13, 14]; Klein argues that much apparent indeterminacy should be understood in terms of isomorphic computational structures rather than as an absence of objective computational facts [23].

1.1 Mechanistic and robust-mapping accounts

Piccinini’s mechanistic theory treats computation as a property of physical mechanisms that manipulate medium-independent vehicles according to rules sensitive to differences among their states [29]. The emphasis shifts from a freely chosen mathematical mapping to organized components, activities, and functional roles. Horsman, Stepney, Wagner, and Kendon similarly analyze physical computation through a representation relation linking an abstract model to a physical evolution and back, with explicit attention to the direction of use and prediction [20].

Anderson and Piccinini’s robust mapping account is the most recent book-length attempt to preserve the useful mapping idea while excluding pathological realizations. A physical system implements a computing system only when it satisfies a robust computational description that captures the physical signature of the computation, not merely a coincidental sequence of state correspondences [2]. The resulting literature converges on a broad lesson: implementation needs structure-preserving constraints stronger than trajectory matching.

What this contributes to the package

The package inherits the anti-triviality lesson but separates two questions that are often fused: what can be said about a *particular actual episode* without counterfactual assumptions, and what stronger claim is required to say that a *mechanism* implements a total computation across alternative cases. This motivates distinct occurrent and modal realization notions.

2 A function is not an algorithm

A second literature addresses a different ambiguity. Two procedures may compute the same mathematical function while differing systematically in internal structure, intermediate states, resources, or method. If “the computation” is identified with the extensional input-output function, these differences disappear by definition.

Moschovakis explicitly treats the foundations of algorithm theory as a problem distinct from computability theory. His recursor-based account records computational structure that is erased when one keeps only the function computed [28]. Gurevich’s Abstract State Machine program similarly seeks an implementation-independent representation of sequential algorithms. The Sequential ASM Thesis is grounded in postulates of sequential time, abstract states, and bounded exploration, and shows that algorithms satisfying these postulates are captured step-for-step by abstract state machines [19].

Blass, Dershowitz, and Gurevich then confront the identity question directly: when are two algorithms the same? Their analysis is valuable partly because it resists the expectation that there is one simple extensional equivalence relation answering every use of “same algorithm” [4]. Algorithm identity depends on what structural features are taken as constitutive.

The elementary example used throughout the companion note captures this point. Summing the integers from 1 to n by iteration and evaluating $n(n+1)/2$ compute the same function. They are nevertheless different procedures. Conversely, when either procedure occurs inside a larger program that sees only the subroutine’s input and output, the larger program may abstract away the difference. This is the first indication that computational identity is relative to an abstraction relation, while the truth of a specified identity claim can remain objective.

Gap exposed

The algorithm literature establishes that algorithms are richer than functions. A general ontology connecting physical episodes, nested local abstractions, modal mechanisms, and families of simultaneously valid computational descriptions additionally requires tools from semantics and

abstraction theory.

3 Operational semantics, transition systems, and bisimulation

Programming-language semantics supplies a mature answer to a question closely related to computational identity: how should one represent the internal dynamics of a program rather than only its denotation? Plotkin’s Structural Operational Semantics represents execution through transitions between configurations, allowing a language’s meaning to be specified by rules governing local steps [30]. This provides a natural home for distinctions between two programs with the same input-output function but different intermediate behavior.

Transition-system theory and process calculi then ask when two dynamic systems should count as behaviorally equivalent. Bisimulation is one of the central answers. Roughly, two states are bisimilar when each transition of one can be matched by a transition of the other in a way that preserves the relation recursively. Coalgebra provides a highly general setting in which state-based systems and behavioral equivalence can be treated uniformly [33]. In compositional settings, equivalence must also be a congruence: replacing a component by an equivalent component should preserve equivalence of the whole. Work such as Leifer and Milner’s derivation of bisimulation congruences makes this substitution principle explicit [24].

These ideas map directly onto the nested-subroutine problem. If two subroutines are equivalent at a specified interface and the enclosing context can interact with them only through that interface, substitution should preserve the abstract behavior of the enclosing system. If the context can inspect timing, memory traces, or intermediate states, the equivalence becomes finer.

What this contributes to the package

The package treats “same computation” not as one primitive relation but as a family of equivalences generated by admissible abstractions. Congruence then becomes the formal condition that makes local abstraction usable inside larger computations.

4 Abstraction, refinement, and non-linear granularity

The natural mathematical language for “forgetting some distinctions while preserving others” is abstraction. Cousot and Cousot’s abstract interpretation develops a lattice-theoretic framework in which concrete and abstract semantic domains are related systematically, often through Galois connections [12]. Although abstract interpretation was created for static analysis rather than metaphysics of computation, its central insight is directly relevant: abstraction is structured information loss, and abstractions themselves can be ordered by precision.

This matters because computational granularity is generally not one-dimensional. If a system has components A and B , one may retain the internals of A while abstracting B , or do the reverse. These two descriptions are usually incomparable. Under independence assumptions, local abstraction orders combine approximately as products, generating the familiar diamond shape of a lattice rather than a single chain from “micro” to “macro”.

The refinement relation is therefore best treated as a partial order. One description refines another when it preserves all the distinctions of the coarser description and possibly more. Meets, joins, or quotient lattices may exist under additional structural assumptions, but they should not be postulated universally.

5 Causal abstraction and intervention-preserving realization

Causal-model abstraction gives a particularly powerful formalization of high-level realization. Rubenstein et al. introduced exact transformations between structural causal models that preserve interventionally relevant predictions across levels [32]. Beckers and Halpern developed a more systematic account of abstraction between causal models, including mappings between low- and high-level variables and interventions [3].

Geiger and collaborators have recently turned causal abstraction into a general foundation for mechanistic interpretability: a high-level computational explanation is faithful when interventions on high-level variables correspond to suitable low-level interventions and the resulting behavior commutes across the abstraction map [17]. Geiger, Harding, and Icard explicitly connect this framework back to classical philosophical questions about computational implementation and representation [18].

This literature strongly supports the modal side of the package’s theory. An intervention-preserving abstraction can say that a physical mechanism implements an abstract computation across alternative cases, not merely along one observed run. It also clarifies why a counterfactual dimension is difficult to avoid when the claim concerns a complete mechanism rather than an actual occurrence.

Caution

Causal abstraction should not be confused with an account of a single actual execution. An intervention-preserving mapping intentionally quantifies over alternative interventions. That is exactly its strength for modal realization, but it is too strong if the target question is only what computational structure was actually instantiated during a finite episode.

6 Actual executions: event structures, occurrence, and dynamic dependence

Concurrency theory provides an object that lies between a bare state sequence and a complete counterfactual machine. Winskel’s event structures represent a computational process by event occurrences together with causal dependency and conflict relations [36]. The key conceptual shift is from a total time-ordered list of global states to a partial order of events: independent events need not be artificially ordered, and causal ancestry can be represented directly.

Dynamic program analysis makes a parallel distinction. Agrawal and Horgan’s dynamic slicing constructs dependence information for a particular execution rather than all possible executions [1]. A dynamic dependence graph records data and control dependencies that actually arise during the run. Such structures are strictly richer than an input-output trace or list of machine snapshots while remaining indexed to one actual history.

Provenance research generalizes the idea of recording how outputs were derived from inputs. Cheney emphasizes that provenance, influence, dependence, explanation, and causality should not be treated as synonyms; causal semantics can be layered on top of provenance structures rather than assumed automatically [11]. This distinction is central to the package. An actual provenance edge can mean that a token was supplied to an operation and used in the physical derivation without yet asserting that the token was a necessary or actual cause of the output under a counterfactual test.

What this contributes to the package

The package’s occurrent object is closest in spirit to a typed, valued provenance/event structure: actual events, actual value tokens, actual production and consumption relations, and partial order. This primitive structure remains distinct from both a complete modal transition system and a philosophical theory of actual causation.

7 Participation, value support, actual causation, and modal dependence

Once an actual execution is represented causally, a new ambiguity appears: what should an edge mean? Consider an OR gate with actual inputs (true, true, false) and output true. All three signals physically arrive at the gate, so all three participate in provenance. Yet the false input does not help establish the true output in the same way as either true input.

Boolean-function complexity provides a precise local notion here. Certificate complexity is a standard decision-tree measure; see the survey by Buhrman and de Wolf [5]. A certificate for the value of a Boolean function at a particular input is a set of input assignments that suffices to force that output regardless of unspecified coordinates; certificate complexity measures the smallest such set. The package generalizes the same idea under the name *value support*. For OR at $(1, 1, 0)$, either singleton assignment $x_1 = 1$ or $x_2 = 1$ is a minimal support for the output 1. For AND at $(1, 1)$, the two true assignments are jointly required.

This notion must be classified carefully. A support is indexed to an actual occurrence and contains only values that actually occurred, but testing sufficiency requires knowledge of the local operation on unrealized inputs. It is therefore locally modal. It is neither primitive provenance nor purely actual causation.

Actual-causation research shows why a further distinction is needed. Halpern-Pearl-style theories and descendants aim to identify which actual events count as causes of a particular outcome, confronting overdetermination and preemption. Provenance work has explicitly studied the relation between derivation graphs and such causal notions [11]. Meliou et al. develop functional causality for database explanations and degrees of responsibility [27]. These approaches demonstrate that “participated”, “was sufficient”, “was necessary”, and “was an actual cause” are mathematically and philosophically distinct.

Gap exposed

A common informal phrase such as “the causal graph of the computation” hides several different relations. The package separates at least: (i) actual provenance/participation, (ii) locally modal value support, (iii) optional actual-contribution relations, and (iv) modal dependence of the mechanism. No single edge type is asked to do all four jobs.

8 Canonical behavioral structures and the return of counterfactuals

There is a natural canonical construction for open deterministic systems: identify states that cannot be distinguished by any future sequence of inputs and observations. This is closely related to automata minimization and Myhill–Nerode-style quotienting. In 2026, Kanai and Ma’s Canonical Functionalism explicitly uses complete future input-output roles to construct a minimal functional state-transition structure intended to avoid observer-relative semantic mappings [21].

The appeal is clear: arbitrary state labels disappear, and equivalent states are identified by their complete behavioral role. The cost is equally clear: the definition is thoroughly counterfactual. State identity depends on all possible future interactions, including those not actually realized.

The package therefore treats canonical modal quotients as important but not exhaustive. They belong to the modal spectrum. A separate occurrent spectrum is needed for claims about a particular realized episode that should not automatically depend on unactualized alternatives.

9 Computational functionalism and consciousness

Computational functionalism about consciousness adds a psychophysical thesis to a theory of computation: some computational or functional organization is supposed to be sufficient, constitutive, or invariant for phenomenology. Chalmers’s organizational invariance and fading/dancing qualia arguments are classic attempts to motivate the preservation of experience under sufficiently fine-grained functional isomorphism [10, 9].

The difficulty is that “functional organization” can mean very different things. It may mean external input-output behavior, a complete modal transition table, a causal network of internal mechanisms, or an actually realized computational process. Thought experiments such as Maudlin’s Olympia target views on which inactive or counterfactual machinery can determine whether a replay-like system is conscious [26]. More recently, the unfolding argument challenges consciousness theories that assign different consciousness to systems with the same input-output behavior solely because their internal causal architecture differs [15]. The argument does not settle which architecture is correct, but it exposes the empirical and conceptual cost of making phenomenology depend on structures that behavior cannot reveal.

The 2026 Intrinsic Computational Functionalism program by Ma and Kanai is directly relevant. It argues that observer-relative mappings are inadequate and that consciousness-relevant computational organization, if any, must be intrinsic to the system’s causal-dynamical structure and invariant under structure-preserving relabelings [25]. Canonical Functionalism proposes a minimal counterfactual behavioral structure [21]; a subsequent paper argues that pure input-output canonicalization is too weak for lookup-table and unfolding cases and enriches the representation with internal mechanisms, interventions, and readouts [22]. These papers are recent preprints and should be treated as an active research program rather than settled consensus.

What this contributes to the package

The companion conceptual note takes a more actualist default than strongly counterfactual functionalism: present phenomenology should depend primarily on invariants of the computation actually generated during the episode. Modal structure remains crucial for capacities and may be added when independently motivated, but off-trajectory differences should not change present phenomenology by themselves.

10 Lookup tables, compression, and resource-sensitive structure

Lookup-table thought experiments expose another distinction. An enormous table can realize the same global state-transition function as a highly structured mechanism while sharing little of its internal decomposition. Function identity therefore does not imply algorithmic or spectral identity.

Complexity theory supplies the general background: a truth table may require exponential description size even when a function has a compact circuit or program. Binary decision diagrams demonstrate how canonical or reduced representations can compress structured Boolean functions dramatically, though worst-case exponential blowups remain [6]. Algorithmic information theory similarly distinguishes raw description length from exploitable regularity. These traditions support a resource-sensitive extension of computational identity: as admissible resource budgets shrink, some implementations disappear, and structural features common to all remaining realizations may become unavoidable.

As an optional extension, this perspective suggests notions such as resource-constrained computational core and spectral rigidity. Compression does not in general force recovery of the original algorithm: distinct short algorithms can compute the same function. Resource constraints force the exploitation of structure, not necessarily a unique structure.

11 A gap map: what is established, and what is synthesis

Problem	Existing mature tools	Synthesis used in this package
Physical implementation	Causal/mapping accounts, mechanistic computation, robust mapping	Admissible realization witnesses separated into occurrent and modal forms
Function vs algorithm	ASM, recursors, algorithm identity literature	Explicit hierarchy from function to algorithm to implementation to run
Actual execution	Event structures, occurrence semantics, dynamic dependence, provenance	Typed valued computational provenance as the primitive occurrent object
Local causal relevance	Certificates, actual causation, responsibility	Separation of participation, value support, actual contribution, and modal dependence
Abstraction	Abstract interpretation, quotienting, bisimulation, congruence	A non-linear spectrum/poset of simultaneously valid computational abstractions
Modal realization	Structural causal models and causal abstraction	Intervention-preserving modal realization as a stronger layer above occurrence
Cross-system comparison	Behavioral equivalence and isomorphism	Spectral containment, equivalence, overlap, and common computational core
Multiplicity	Subsystem semantics and compositionality	Witness-aware realization structure rather than a spectrum treated as a bare set
Consciousness use	Organizational invariance, functionalism, ICF, IIT-related debates	Constraints on the <i>form</i> of admissible computational functionalism, especially actual-process and counterfactual-irrelevance principles

The main novelty is therefore architectural rather than the invention of every mathematical ingredient. The literature already contains powerful theories of implementation, algorithms, abstraction, causality, event occurrence, and behavioral equivalence. What is less standard is to place them in one layered ontology in which:

1. actual execution is represented more richly than a state sequence but less strongly than a counterfactual mechanism;
2. modal implementation is a separate strengthening rather than the definition of all computation;
3. abstractions form a structured family rather than a single privileged level;
4. computational facts are attached to realization witnesses, preserving multiplicity and locality;
5. value support and actual causation are optional enrichments rather than silently conflated with provenance;
6. computational functionalism is constrained by invariance principles inherited from this ontology.

12 Recommended reading path

For physical implementation and triviality, begin with Chalmers [7, 8], then Piccinini [29], Sprevak [35], and Anderson–Piccinini [2]. For algorithm identity, read Moschovakis [28], Gurevich [19], and Blass–Dershowitz–Gurevich [4]. For abstraction, pair Cousot–Cousot [12] with Beckers–Halpern [3] and Geiger et al. [17]. For actual execution, Winskel [36], Agrawal–Horgan [1], and Cheney [11] provide complementary views. For the consciousness-facing edge of the debate, read Chalmers on or-

ganizational invariance, Maudlin’s Olympia, the unfolding argument, and the 2026 intrinsic/canonical functionalism papers [26, 15, 25, 21, 22].

Bibliographic note

The 2026 papers by Kanai, Ma, Geiger, Harding, Icard, and collaborators are recent work and, where indicated below, preprints. They are included because they directly address computational individuation, causal abstraction, and computational functionalism in forms unusually close to the questions of this package, and are treated here as an active research program rather than field-wide consensus.

References

- [1] H. Agrawal and J. R. Horgan. Dynamic program slicing. *Proceedings of PLDI*, 1990, pp. 246–256.
- [2] N. G. Anderson and G. Piccinini. *The Physical Signature of Computation: A Robust Mapping Account*. Oxford University Press, 2024.
- [3] S. Beckers and J. Y. Halpern. Abstracting causal models. *Proceedings of AAAI*, 33:2678–2685, 2019.
- [4] A. Blass, N. Dershowitz, and Y. Gurevich. When are two algorithms the same? *Bulletin of Symbolic Logic*, 15(2):145–168, 2009.
- [5] H. Buhrman and R. de Wolf. Complexity measures and decision tree complexity: a survey. *Theoretical Computer Science*, 288(1):21–43, 2002.
- [6] R. E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986.
- [7] D. J. Chalmers. On implementing a computation. *Minds and Machines*, 4:391–402, 1994.
- [8] D. J. Chalmers. Does a rock implement every finite-state automaton? *Synthese*, 108:309–333, 1996.
- [9] D. J. Chalmers. Absent qualia, fading qualia, dancing qualia. In *Conscious Experience*, 1995; revised material appears in *The Conscious Mind*, 1996.
- [10] D. J. Chalmers. *The Conscious Mind*. Oxford University Press, 1996.
- [11] J. Cheney. Causality and the semantics of provenance. arXiv:1004.3241, 2010.
- [12] P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. *POPL*, pp. 238–252, 1977.
- [13] A. Curtis-Trudel. Implementation as resemblance. *Philosophy of Science*, 88(5):1021–1032, 2021.
- [14] A. Curtis-Trudel. The determinacy of computation. *Synthese*, 200, 2022.
- [15] A. Doerig, A. Schurger, K. Hess, and M. H. Herzog. The unfolding argument: why IIT and other causal structure theories cannot explain consciousness. *Consciousness and Cognition*, 72:49–59, 2019.
- [16] N. Fresco, B. J. Copeland, and M. J. Wolf. The indeterminacy of computation. *Synthese*, 199:12753–12775, 2021.

- [17] A. Geiger et al. Causal abstraction: a theoretical foundation for mechanistic interpretability. *Journal of Machine Learning Research*, 26(83):1–64, 2025.
- [18] A. Geiger, J. Harding, and T. Icard. How causal abstraction underpins computational explanation. arXiv:2508.11214, revised 2026.
- [19] Y. Gurevich. Sequential Abstract-State Machines capture sequential algorithms. *ACM Transactions on Computational Logic*, 1(1):77–111, 2000.
- [20] C. Horsman, S. Stepney, R. C. Wagner, and V. Kendon. When does a physical system compute? *Proceedings of the Royal Society A*, 470:20140182, 2014.
- [21] R. Kanai and S. Ma. Canonical Functionalism: defining functional structure without observer-relative semantic maps. arXiv:2605.21506, 2026.
- [22] R. Kanai and S. Ma. Intrinsic Computational Functionalism and simulated consciousness. arXiv:2606.15348, 2026.
- [23] C. Klein. Computational individuation: isomorphism, not indeterminacy. *Analysis*, 86(1):60–70, 2026. doi:10.1093/analys/anaf003.
- [24] J. J. Leifer and R. Milner. Deriving bisimulation congruences for reactive systems. In *CONCUR 2000*, LNCS 1877, pp. 243–258.
- [25] S. Ma and R. Kanai. Intrinsic Computational Functionalism: from observer-relative maps to observer-independent structures. arXiv:2606.06424, 2026.
- [26] T. Maudlin. Computation and consciousness. *Journal of Philosophy*, 86(8):407–432, 1989.
- [27] A. Meliou, W. Gatterbauer, K. F. Moore, and D. Suciu. Why so? or Why no? Functional causality for explaining query answers. arXiv:0912.5340, 2009 (conference version 2010).
- [28] Y. N. Moschovakis. What is an algorithm? In *Mathematics Unlimited – 2001 and Beyond*, Springer, pp. 919–936, 2001.
- [29] G. Piccinini. *Physical Computation: A Mechanistic Account*. Oxford University Press, 2015.
- [30] G. D. Plotkin. A structural approach to operational semantics. DAIMI FN-19, Aarhus University, 1981.
- [31] H. Putnam. *Representation and Reality*. MIT Press, 1988.
- [32] P. K. Rubenstein et al. Causal consistency of structural equation models. *UAI*, 2017.
- [33] J. J. M. M. Rutten. Universal coalgebra: a theory of systems. *Theoretical Computer Science*, 249(1):3–80, 2000.
- [34] M. Sprevak. Three challenges to Chalmers on computational implementation. *Journal of Cognitive Science*, 13(2):107–143, 2012.
- [35] M. Sprevak. Triviality arguments about computational implementation. In *The Routledge Handbook of the Computational Mind*, 2018.
- [36] G. Winskel. Event structures. In *Petri Nets: Applications and Relationships to Other Models of Concurrency*, LNCS 255, Springer, 1987.